

Service Boundary – Brief Summary

A **service boundary** defines what responsibilities, data, and business capability belong inside a microservice.

Key Principles

- **Business Capability Based**
 - Each service represents one clear business function.
- **Owns Its Data**
 - A service has its own database.
 - No direct DB access from other services.
- **High Cohesion**
 - All functionalities inside the service are closely related.
- **Low Coupling**
 - Avoid excessive synchronous calls between services.
- **Independent Deployment**
 - Services should be deployable without coordinating with others.
- **Change Isolation**
 - Features that change together → same service.
 - Features that evolve independently → separate services.
- **Transaction Boundary**
 - Avoid frequent distributed transactions.
 - Prefer eventual consistency (Saga pattern).

One-Line Definition

> A service boundary defines the scope of responsibility and data ownership of a microservice to ensure high cohesion, low coupling, and independent scalability.

Custom Aspect

 `</> Java`

```
import java.lang.annotation.*;

@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
@Documented
public @interface CheckRole {
    String value(); // role required
}
```

4 Use Annotation on Method

 `</> Java`

```
import org.springframework.stereotype.Service;

@Service
public class OrderService {

    @CheckRole("ADMIN")
    public void createOrder() {
        System.out.println("Order Created");
    }
}
```

JDBC

2. JDBC Repository (Using Plain JDBC)

 Java 

```
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class UserRepository {

    private static final String URL = "jdbc:mysql://localhost:3306/testdb";
    private static final String USER = "root";
    private static final String PASSWORD = "password";

    public List<UserDTO> getAllUsers() {

        List<UserDTO> users = new ArrayList<>();
        String query = "SELECT id, name, email, status FROM users";

        try (
            Connection connection = DriverManager.getConnection(URL, USER, PASSWORD);
            PreparedStatement ps = connection.prepareStatement(query);
            ResultSet rs = ps.executeQuery()
        ) {

            while (rs.next()) {
                UserDTO user = new UserDTO(
                    rs.getLong("id"),
                    rs.getString("name"),
                    rs.getString("email"),
                    rs.getString("status")
                );
                users.add(user);
            }

        } catch (SQLException e) {
            throw new RuntimeException("Error fetching users", e);
        }

        return users;
    }
}
```

Dynamic rules engine

Dynamic Rule Engine (Teammate+) – Complete Interview Explanation

1. Overview

In Teammate+, a **dynamic rule engine** is implemented to allow business logic to be configured without modifying application code. Instead of hardcoding conditions in Java services, rules are created through an **admin UI page**, stored in the **database**, and evaluated dynamically at runtime.

This design enables business teams or administrators to change system behavior without redeploying the application.

Typical use cases:

- Assessment validation rules
- Risk classification based on score
- Conditional workflow triggers
- Approval requirements
- Dynamic scoring logic
- Conditional notifications
- Role-based workflow actions

2. Problem With Hardcoded Business Logic

Without a rule engine, business logic is embedded directly in code.

Example:

```
```java
if(score < 40){
 assessment.setRiskLevel("HIGH");
}
```
```

Problems with this approach:

- Requires code changes for every rule update
- Requires deployment
- Business users cannot manage logic
- Difficult to scale rule complexity

A dynamic rule engine externalizes this logic into the database.

3. Core Idea of Dynamic Rule Engine

Rules are stored as ****data instead of code****.

Example rule stored in DB:

| rule_name | condition_expression | action_expression |
|----------------|----------------------|----------------------|
| HIGH_RISK_RULE | score < 40 | setRiskLevel("HIGH") |

At runtime:

1. Rules are loaded from the database
2. Conditions are evaluated dynamically
3. If the condition is true, the action is executed

4. High Level Architecture

Admin UI
↓
Rule Management API
↓
Database (Rules Table)
↓
Rule Engine Service
↓
Load Rules
↓
Evaluate Conditions
↓
Execute Actions
↓
Return Modified Entity

5. Rule Creation Flow

Step 1 — Admin Creates Rule From UI

| Field | Value |
|-------------|----------------------|
| Rule Name | High Risk Assessment |
| Entity Type | Assessment |
| Condition | score < 40 |
| Action | setRiskLevel("HIGH") |
| Priority | 10 |

| Active | true |

Step 2 — Rule Stored In Database

Example table:

```
```sql
CREATE TABLE rules (
 id BIGINT PRIMARY KEY,
 rule_name VARCHAR(255),
 entity_type VARCHAR(100),
 condition_expression TEXT,
 action_expression TEXT,
 priority INT,
 is_active BOOLEAN,
 created_at TIMESTAMP,
 updated_at TIMESTAMP
);
```
```

Example record:

| id | rule_name | entity_type | condition_expression | action_expression | priority | is_active |
|----|----------------|-------------|----------------------|----------------------|----------|-----------|
| 1 | HIGH_RISK_RULE | ASSESSMENT | score < 40 | setRiskLevel("HIGH") | 10 | true |

6. Rule Execution Flow

When an event occurs (example: assessment submission):

Assessment Submitted

↓

Rule Engine Service

↓

Fetch Active Rules From Database

↓

Filter Rules By Entity Type

↓

Sort Rules By Priority

↓

Evaluate Condition

↓

Execute Action

↓

Return Updated Object

7. Core Components

Rule Entity

```
```java
public class Rule {

 private Long id;
 private String ruleName;
 private String entityType;
 private String conditionExpression;
 private String actionExpression;
 private Integer priority;
 private Boolean active;
}
```
```

Rule Repository

```
```java
@Repository
public interface RuleRepository extends JpaRepository<Rule, Long> {

 List<Rule> findByEntityTypeAndActiveTrueOrderByPriorityDesc(String entityType);
}
```
```

Ensures:

- Only active rules are loaded
- Rules executed in priority order

Rule Engine Service

```
```java
@Service
public class RuleEngineService {

 @Autowired
 private RuleRepository ruleRepository;
}
```

```

public void evaluateRules(Assessment assessment){

 List<Rule> rules = ruleRepository
 .findByEntityTypeAndActiveTrueOrderByPriorityDesc("ASSESSMENT");

 for(Rule rule : rules){

 if(evaluateCondition(rule.getConditionExpression(), assessment)){
 executeAction(rule.getActionExpression(), assessment);
 }
 }
}
}
...

```

## ## 8. Condition Evaluation

Conditions stored as expressions.

Examples:

```

...
score < 40
department == 'Finance'
riskScore > 80
assessmentType == 'INTERNAL'
...
```

Evaluated using **\*\*Spring Expression Language (SpEL)\*\***.

```

```java
private boolean evaluateCondition(String expression, Assessment assessment){

    ExpressionParser parser = new SpelExpressionParser();

    EvaluationContext context = new StandardEvaluationContext(assessment);

    Boolean result = parser.parseExpression(expression).getValue(context, Boolean.class);

    return Boolean.TRUE.equals(result);
}
...

---
```

9. Action Execution

Actions executed when condition is satisfied.

Examples:

```
...  
setRiskLevel("HIGH")  
setApprovalRequired(true)  
assignReviewer("RISK_TEAM")  
sendNotification("ADMIN")  
...
```

Example implementation:

```
```java  
private void executeAction(String action, Assessment assessment){

 if(action.contains("setRiskLevel")){
 assessment.setRiskLevel("HIGH");
 }

 if(action.contains("setApprovalRequired")){
 assessment.setApprovalRequired(true);
 }
}
...`
```

In advanced systems actions use **strategy pattern** or **handler classes**.

---

## ## 10. Rule Caching

Fetching rules from the database every request is inefficient.

Rules are cached in memory.

Application Startup

↓

Load Rules From Database

↓

Store In Memory Cache

↓

Runtime Rule Evaluation

Example cache:

```
...
Map<String, List<Rule>> ruleCache
...
```

Example:

```
...
ASSESSMENT -> [Rule1, Rule2, Rule3]
SURVEY -> [Rule4, Rule5]
...
```

Cache refresh strategies:

- Scheduled refresh
- Event-based refresh
- Manual refresh

---

## ## 11. Rule Priority Handling

Multiple rules may apply.

Rule	Condition	Action
HIGH_RISK_RULE	score < 40	setRiskLevel("HIGH")
CRITICAL_RISK_RULE	score < 30	setRiskLevel("CRITICAL")

Priority:

Rule	Priority
CRITICAL_RISK_RULE	20
HIGH_RISK_RULE	10

Higher priority executes first.

---

## ## 12. Rule Versioning

Track rule updates.

```
```sql
CREATE TABLE rule_versions (
  id BIGINT PRIMARY KEY,
  rule_id BIGINT,
  version INT,

```

```

        condition_expression TEXT,
        action_expression TEXT,
        created_at TIMESTAMP
    );
    ...

```

Benefits:

- Audit history
- Rollback capability
- Track rule evolution

13. Rule Execution Logging

```

```sql
CREATE TABLE rule_execution_log (
 id BIGINT PRIMARY KEY,
 rule_id BIGINT,
 entity_id BIGINT,
 result VARCHAR(50),
 executed_at TIMESTAMP
);
...

```

Example log:

```

| rule_id | entity_id | result |
|-----|-----|-----|
| 1 | 4521 | SUCCESS |

```

---

## ## 14. Example End-to-End Scenario

Input:

```

...
score = 35
department = Finance
...

```

Rules:

```

| Rule | Condition | Action |
|----|-----|-----|
| Rule1 | score < 40 | setRiskLevel("HIGH") |

```

```
| Rule2 | department == 'Finance' | setApprovalRequired(true) |
```

Execution:

```
...
```

```
Rule1 triggered → riskLevel = HIGH
```

```
Rule2 triggered → approvalRequired = true
```

```
...
```

Final result:

```
...
```

```
riskLevel = HIGH
```

```
approvalRequired = true
```

```
...
```

```

```

### ## 15. Advantages

- Dynamic business logic
- No deployment required
- Business-friendly configuration
- Centralized rule management
- Extensible architecture

```

```

### ## 16. Limitations

- Complex debugging
- Rule conflicts
- Performance concerns for large rule sets

```

```

### ## 17. Interview Summary Answer

In Teammate+, a dynamic rule engine externalizes business logic from application code.

Administrators create rules via a UI where each rule contains a condition and action. These rules are stored in the database and evaluated dynamically at runtime using Spring Expression Language. The rule engine loads active rules, evaluates conditions, executes actions, and applies them to entities.

Rules are prioritized and cached in memory to improve performance. This allows business logic to change without modifying code or redeploying the application.

# Hackthon



## Hackathon Project: AI-Driven Risk Auditing(Code Games 2024 – 1st Place)

### Summary:

Built an AI-powered enterprise risk intelligence platform to assist auditors in identifying similar historical risks and automatically generating mitigation controls. The system leveraged semantic vector search and large language models to improve risk discovery beyond traditional keyword-based search systems.

### Core Technologies Used:

- Backend: Java, Spring Boot (REST APIs)
- AI/LLM: OpenAI GPT-3.5 Turbo
- Embeddings: OpenAI Text Embeddings
- Vector Database: Pinecone (vector embeddings)
- Frontend: React
- Architecture: RAG-style semantic retrieval + LLM generation

RAG -> retrieval augmented generation

### Technical Implementation:

#### 1. Semantic Risk Representation

Risk descriptions were converted into high-dimensional vector embeddings using OpenAI embedding models.

These embeddings capture semantic meaning, allowing detection of risks that are conceptually similar even when the wording differs.

#### 2. Vector Storage and Retrieval

Embeddings were stored in the Pinecone vector database.

For each new risk submitted by an auditor:

- The risk text was converted to an embedding
- A similarity search (cosine similarity) was executed in Pinecone
- The system retrieved the top K most semantically similar historical risks

#### 3. Semantic Risk Search Pipeline

New Risk Input



Embedding Generation (OpenAI API)



Vector Query in Pinecone



Top-K Similar Historical Risks



Context sent to LLM

#### 4. AI Risk and Control Generation

The retrieved risks were used as context for GPT-3.5 Turbo to generate:

- New potential lookalike risks
- Recommended mitigation controls
- Compliance suggestions

Prompt structure included:

- current risk
- similar historical risks
- control examples

## 5. Backend API Flow

POST /risk/analyze

Steps:

1. Receive risk description
2. Generate embedding via OpenAI API
3. Query Pinecone for similar vectors
4. Send retrieved risks to GPT-3.5
5. Return generated risks and mitigation controls

## 6. Key Technical Concepts Implemented

Vector Embeddings

Transforms text into numeric vectors that represent semantic meaning.

Approximate Nearest Neighbor Search

Used by Pinecone to efficiently find similar vectors among large datasets.

Semantic Retrieval

Enables contextual similarity search rather than keyword matching.

LLM-Based Risk Generation

GPT-3.5 generated additional risks and suggested mitigation strategies using retrieved context.

## 7. System Outcome

The platform enabled:

- Semantic search across historical audit risks
- AI-generated risk discovery
- Automated control recommendations

This significantly improved the efficiency of risk analysis and audit preparation, which led to the project winning **\*\*1st place in the ESG category at the Code Games 2024 hackathon\*\***.

Kafka

topics

- 1) DLQueue
- 2) retry mechanism
- 3) fan out

=====

## KAFKA SHORT NOTES

=====

### 1) TOPICS AND THEIR USE

-----

- Topic = Logical category where messages are stored.
- Producer publishes to a topic.
- Consumers subscribe to a topic.
- Topic is divided into partitions.

Example:

order.created  
order.payment.completed

Use:

- Decouple services
- Enable async communication
- Event-driven architecture

### 2) PUBLISHER SMALL SETUP (SPRING BOOT)

-----

application.properties

spring.kafka.bootstrap-servers=localhost:9092

Producer Service

@Autowired

private KafkaTemplate<String, Object> kafkaTemplate;

```
public void publishOrder(Order order) {
 kafkaTemplate.send("order.created", order);
}
```

Flow:

Order Object

↓  
JsonSerializer  
↓  
Kafka Broker  
↓  
Stored in Topic Partition

### 3) CONSUMER SMALL SETUP (SPRING BOOT)

---

application.properties

spring.kafka.bootstrap-servers=localhost:9092

Consumer Listener

```
@KafkaListener(
 topics = "order.created",
 groupId = "payment-group"
)
public void consumeOrder(Order order) {
 System.out.println(order);
}
```

Flow:

Consumer joins group

↓

Kafka assigns partition

↓

Polls messages

↓

Processes message

↓

Commits offset

### 4) PARTITIONING AND OFFSET

---

Partition:

- Topic is divided into partitions.
- Enables parallel processing.
- Ordering guaranteed only inside one partition.

Example:

Topic: order.created

Partition 0:

Offset 0 → Order1

Offset 1 → Order4

Partition 1:

Offset 0 → Order2

Offset 1 → Order5

Offset:

- Sequential number inside partition.
- Tracks consumer progress.
- Stored per (Consumer Group + Partition).

Rule:

Next read = Last committed offset + 1

## 5) SMALL END-TO-END FLOW

---

User places order



Order Service



Publish → order.created topic



Kafka stores message in Partition



Payment Service consumes



Payment successful



Publish → order.payment.completed



Delivery Service consumes

## 6) ZOOKEEPER AND KAFKA SERVER (LOCAL)

---

Old Setup:

ZooKeeper + Kafka Broker

ZooKeeper:

- Managed metadata
- Leader election
- Broker coordination

Kafka Broker:

- Stores messages
- Handles partitions
- Serves producers & consumers

Modern Setup (Recommended):

Kafka in KRaft mode (No ZooKeeper)

Local:

- Run Kafka via Docker
- Expose port 9092
- Connect via localhost:9092

=====  
KEY FORMULA  
=====

Partition → Parallelism

Consumer Group → Load balancing

Offset → Progress tracking

Kafka → Event backbone

=====

=====  
1) RETRY MECHANISM IN KAFKA  
=====

Retry is used when message processing fails temporarily  
(DB issue, API failure, network error).

Instead of losing the message, the consumer retries processing.

Flow:

Producer  
↓  
Kafka Topic  
↓  
Consumer reads message  
↓  
Processing fails  
↓  
Retry message  
↓  
If retry limit reached → DLQ

## Common Retry Approaches

### 1) Immediate Retry

- Retry inside consumer code.
- Not recommended for heavy workloads.

### 2) Retry Topics (Production Pattern)

```
order.created
 ↓ fail
order.created.retry.5s
 ↓ fail
order.created.retry.1m
 ↓ fail
order.created.dlq
```

### 3) Spring Kafka Retry

```
@RetryableTopic(
 attempts = 3,
 backoff = @Backoff(delay = 5000)
)
```

Used when failures are temporary.

## =====

### 2) OFFSET COMMITTING

## =====

Offset = position of a message inside a partition.

Kafka uses offset to track which messages a consumer has processed.

Structure:

Consumer Group + Partition + Offset

Example:

Partition 0

Offset 0 → Order1

Offset 1 → Order2

Offset 2 → Order3

If offset 1 committed → next read starts from offset 2.



## Types of Offset Commit

### 1) Auto Commit

`enable.auto.commit=true`

Kafka automatically commits offsets periodically.

Risk:

Message may be lost if processing fails after commit.

### 2) Manual Commit (Recommended)

Process message

↓

Commit offset

Spring Example:

`ack.acknowledge()`

Ensures message processed before committing offset.

## =====

### 3) DLQ (DEAD LETTER QUEUE)

## =====

DLQ stores messages that cannot be processed even after multiple retries.

Purpose:

- Prevent message loss
- Allow debugging
- Prevent blocking consumer

Flow:

`order.created`

↓ fail

Retry Topic

↓ fail

Retry Topic

↓ fail

`order.created.dlq`

DLQ message usually contains:

- original event
- error reason
- retry count
- timestamp

Engineers later inspect DLQ and decide to:

- reprocess
- fix data
- discard message

#### =====

#### 4) EVENT ID IN KAFKA

#### =====

Event ID is a unique identifier for each message.

Kafka itself does NOT generate event IDs.  
The producer service generates them.

Usually generated using:

UUID.randomUUID()

Example event:

```
{
 "eventId": "a1b2c3",
 "orderId": "ORD123",
 "status": "CREATED"
}
```

Purpose:

- prevent duplicate processing
- track events
- enable idempotency

#### =====

#### 5) DUPLICATE EVENT ID HANDLING

#### =====

Kafka can sometimes deliver duplicate messages due to:

- consumer retry

- network failure
- rebalance
- producer retry

To avoid processing duplicates, consumers implement IDEMPOTENCY.

Approach:

Before processing event:

Check if eventId is already processed.

Flow:

```

Consumer receives event
 ↓
Check eventId in storage
 ↓
If exists → ignore
If not exists → process event
 ↓
Store eventId

```

Where eventId is stored:

1) Database table

```

processed_events

event_id
processed_at

```

2) Redis (for high throughput)

```

key = processed_event:{eventId}
TTL = 24 hours

```

Example logic:

```

if(eventId exists)
 skip processing
else
 process event
 store eventId

```

=====